
```
import random
entry_i = center_i
entry_j = center_j
where_we_can_go = ['up_left', 'up_right', 'down_left', 'down_right']
where_to_go = random.choice(where_we_can_go)
```

Here, we've used Python's `random.choice()` function, which does random selection from lists. It takes an element from the set we specified (`where_we_can_go`), but it chooses at random (or as close to random as it can get).

After we've decided a direction to travel, we can follow whichever rule corresponds to our direction of travel. If we have chosen to go `down_left` or `up_right`, we'll follow Rule 1, choosing the right arguments and indices as follows:

```
if(where_to_go == 'up_right'):
    new_entry_i = entry_i - 1
    new_entry_j = entry_j + 1
    square[new_entry_i][new_entry_j] = rule1(square[entry_i][entry_j],n,True)

if(where_to_go == 'down_left'):
    new_entry_i = entry_i + 1
    new_entry_j = entry_j - 1
    square[new_entry_i][new_entry_j] = rule1(square[entry_i][entry_j],n,False)
```

Similarly, we'll follow Rule 2 if we have chosen to travel `up_left` or `down_right`:

```
if(where_to_go == 'up_left'):
    new_entry_i = entry_i - 1
    new_entry_j = entry_j - 1
    square[new_entry_i][new_entry_j] = rule2(square[entry_i][entry_j],n,True)

if(where_to_go == 'down_right'):
    new_entry_i = entry_i + 1
    new_entry_j = entry_j + 1
    square[new_entry_i][new_entry_j] = rule2(square[entry_i][entry_j],n,False)
```

This code is for going up-left and down-right, but we should follow it only if we're not crossing the antidiagonal. We'll have to make sure that we follow Rule 3 in the case where we are crossing the antidiagonal. There is a